

Arrays and Functions

Discussing Arrays and Functions through Code Snippets and Knowledge Base

Subject: Programming for Problem Solving (ECS201) • **Semester:** 2 • **Department:** Computer Science & Engineering • **Faculty:** Dr. A. Sharma • **Date:** August 3, 2026

1. Introduction to Arrays

An **array** is a contiguous collection of elements of the same data type stored sequentially in memory. Arrays allow constant-time $O(1)$ index-based random access.

Definition (Array):

An array is a fixed-size, homogeneous data structure that stores elements in contiguous memory locations.

□ Best Practice

Always ensure that array access stays within bounds `[0, size - 1]` to avoid undefined behavior or segmentation faults in languages like C/C++.

★ **Key Takeaway:** Array indices start at `0` in zero-indexed languages. The address of element `arr[i]` is computed as:

$$\text{Address}(\text{arr}[i]) = \text{Base Address} + i \times \text{sizeof}(\text{element})$$

Code Example: Dynamic vs Static Allocation

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n = 5;
    // Dynamic memory allocation for array
    int *arr = (int *)malloc(n * sizeof(int));
    if (arr == NULL) return 1;

    for (int i = 0; i < n; i++) {
        arr[i] = (i + 1) * 10;
        printf("arr[%d] = %d\n", i, arr[i]);
    }
}
```

```
}  
  
free(arr);  
return 0;  
}
```

2. Time Complexity & Theoretical Foundations

Theorem 1 (Time Complexity of Linear Search):

Given an un-ordered array of size N , searching for a target element using Linear Search requires $O(N)$ comparisons in the worst case.

Proof: In the worst-case scenario, the target element is located at the last index $N - 1$ or is not present in the array at all. The algorithm must inspect every element from index 0 up to $N - 1$, performing exactly N comparisons. Thus, the time complexity is $O(N)$. □

Corollary (Sorted Array Search):

If the array is sorted, Binary Search reduces the search time complexity to $O(\log N)$.

Example 1 (Array Access Efficiency):

Consider a 1D array `int arr[1000]`. Accessing `arr[42]` takes $O(1)$ time because the memory address is calculated directly via pointer arithmetic.

3. Summary Comparison

Operation	Static Array	Dynamic Array	Notes
Index Access	$O(1)$	$O(1)$	Direct pointer offset calculation
Insertion at End	$O(1)^*$	$O(1)$ amortized	Requires space check for static
Search (Unsorted)	$O(N)$	$O(N)$	Requires full linear scan
Search (Sorted)	$O(\log N)$	$O(\log N)$	Binary search algorithm

⚠ Memory Allocation Note

Un-freed dynamic allocations result in memory leaks. Always pair `malloc()` / `calloc()` with `free()`.

📌 **TODO:** Add section on multi-dimensional array mapping functions in column-major vs row-major order.

4. Practice Questions & Knowledge Base

Question 1

5 pts

What is the primary advantage of arrays over linked lists regarding memory locality?

Answer:

Arrays store elements in contiguous memory locations, which maximizes CPU cache hit ratios due to spatial locality of reference.

Question 2

5 pts

Explain why passing a large array to a function in C by value is inefficient.

Answer:

Passing by value copies the entire array contents onto the call stack, causing $O(N)$ memory copy overhead. Passing by reference (pointer) passes only the 8-byte memory address.

Question 3

10 pts

Write a function signature in C to accept a 2D array with 4 columns.

Answer:

```
void processMatrix(int matrix[][4], int rows);
```

Quick Self-Check Q&A

Q1: What is the time complexity of inserting an element at the beginning of an array?

A: $O(N)$ because all existing elements must be shifted one index to the right.

Q2: What does `sizeof(arr) / sizeof(arr[0])` evaluate to for a statically allocated array?

A: It evaluates to the total number of elements in the static array.